

TRD

Technical Requirements Document (TRD)

AWS Glue Data Pipeline for Customer Order Processing

•

1. Document Overview

Purpose

This Technical Requirements Document translates the functional requirements into implementation-ready technical specifications for an AWS Glue-based data pipeline. It defines the technical architecture, data structures, transformation logic, and operational parameters required to build a production-grade solution that ingests customer and order data, performs data quality operations, implements SCD Type 2 using Apache Hudi, and produces analytical aggregations.

Scope

This TRD covers:

- AWS Glue job configuration and execution parameters
- Source-to-target data mappings with schema definitions
- Data quality and transformation logic implementation details
- Apache Hudi configuration for SCD Type 2 implementation
- AWS Glue Data Catalog integration specifications
- S3 storage patterns and file format specifications
- Error handling and logging strategies
- Incremental processing implementation approach
- Aggregation computation logic

This TRD does NOT cover:

- IAM role creation or policy definitions
- AWS Glue job scheduling or orchestration configuration
- Infrastructure-as-Code templates (CloudFormation/Terraform)
- Cost optimization strategies
- Performance tuning beyond standard configurations
- Backup and disaster recovery procedures

Assumptions

1. AWS Glue version 4.0 will be used with Python Shell or Spark engine
2. CSV files use comma (`,`) as delimiter with header row present

3. String "Null" (case-insensitive) and actual NULL values both represent missing data
4. Data types for customer and order schemas will be inferred from CSV content or explicitly cast as follows:
 - CustId: String
 - Name: String
 - EmailId: String
 - Region: String
 - OrderId: String
 - ItemName: String
 - PricePerUnit: Decimal(10,2)
 - Qty: Integer
 - Date: Date (format to be determined at runtime)
 - TotalAmount: Decimal(12,2)
5. Duplicate detection uses exact match across all columns (no fuzzy matching)
6. SCD2 change detection compares all business attributes; key column for matching is CustId + OrderId combination
7. Hudi table will use Copy-on-Write (CoW) storage type with default configurations
8. Hudi record key will be a composite of CustId and OrderId
9. Hudi precombine field will be OpTs (operation timestamp)
10. Incremental processing baseline will be stored as a checkpoint file in S3 or maintained via Hudi commit timeline
11. Date parsing will attempt multiple common formats (YYYY-MM-DD, MM/DD/YYYY, DD-MM-YYYY)
12. Aggregation grouping by "Date" refers to the order date, not a processing date
13. Glue Data Catalog database `gen\_ai\_poc\_databrickscoe` exists prior to job execution
14. S3 bucket `adif-sdlc` exists with appropriate folder structure
15. Glue job will run with sufficient DPU allocation (minimum 2 DPUs for standard jobs, 0.0625 DPU for Python Shell)
16. Parquet format will be used for catalog-registered tables (customer, order) unless otherwise specified
17. Hudi format will be used exclusively for ordersummary table
18. Parquet format will be used for customeraggregatespend analytics table
19. No data encryption requirements specified; default S3 encryption will be used
20. No specific partitioning strategy specified; will implement date-based partitioning where applicable
21. Overwrite mode will be used for catalog tables; upsert mode for Hudi table
22. CloudWatch Logs will be used for job logging with default retention period

-

2. Technical Requirements

TR-INGEST-001: Technical Implementation for Customer Data Ingestion

- Related Functional Requirement ID(s): FR-INGEST-001

- Objective:

Implement S3 CSV reader to load customer data into a Spark DataFrame with schema validation and error handling.

- Source Details:
- Source Type: AWS S3 CSV Files
- Source Path: s3://adif-sdlc/sdlc_wizard/customerdata/
- Source Format: CSV with header
- Source Delimiter: Comma (`,`)
- Source Encoding: UTF-8 (assumed)

- Target Details:
- Target Type: In-memory Spark DataFrame
- Target Name: df_customer_raw

- Input Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	CustId	String	Variable	No	
	Name	String	Variable	No	
	EmailId	String	Variable	No	
	Region	String	Variable	No	

- Output Schema:

Same as Input Schema (in-memory DataFrame)

- Processing / Transformation Logic:
 1. Use AWS Glue DynamicFrame or Spark DataFrame reader to read CSV files
 2. Enable schema inference or apply explicit schema definition
 3. Set header option to true
 4. Set delimiter to comma
 5. Enable multiLine option if records may span multiple lines
 6. Load all files matching pattern `s3://adif-sdlc/sdlc\_wizard/customerdata/.csv`

7. Validate that all expected columns (CustId, Name, EmailId, Region) are present

8. Store result in DataFrame named df_customer_raw

- Validation Rules:

- Verify S3 path exists and is accessible

- Verify at least one CSV file is present in the path

- Verify CSV header contains exactly: CustId, Name, EmailId, Region (order may vary)

- Verify at least one data row exists after header

- Error Handling and Logging:

- Log INFO: "Starting customer data ingestion from s3://adif-sdlc/sdlc_wizard/customerdata/"

- Log INFO: "Number of files found: {count}"

- Log INFO: "Number of raw records loaded: {count}"

- If S3 path does not exist: Log ERROR and raise exception "Source path not found: s3://adif-sdlc/sdlc_wizard/customerdata/"

- If no CSV files found: Log ERROR and raise exception "No CSV files found in source path"

- If schema mismatch: Log ERROR and raise exception "CSV schema does not match expected columns"

- If read operation fails: Log ERROR with exception details and halt job

- Runtime Parameters:

- `source\_customer\_path`: s3://adif-sdlc/sdlc_wizard/customerdata/

- `csv\_delimiter`: `,`

- `csv\_header`: `true`

- `csv\_infer\_schema`: `true`

- Operational Notes:

- Use Glue DynamicFrame for better integration with Glue Data Catalog

- Consider using pushdown predicates if filtering is needed at read time

- Monitor CloudWatch metrics for read throughput and data volume

-

TR-INGEST-002: Technical Implementation for Order Data Ingestion

- Related Functional Requirement ID(s): FR-INGEST-002

- Objective:

Implement S3 CSV reader to load order data into a Spark DataFrame with schema validation and error handling.

- Source Details:
- Source Type: AWS S3 CSV Files
- Source Path: s3://adif-sdlc/sdlc_wizard/orderdata/
- Source Format: CSV with header
- Source Delimiter: Comma (`,`)
- Source Encoding: UTF-8 (assumed)

- Target Details:
- Target Type: In-memory Spark DataFrame
- Target Name: df_order_raw

- Input Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	OrderId	String	Variable	No	
	ItemName	String	Variable	No	
	PricePerUnit	Decimal	10,2	No	
	Qty	Integer	-	No	
	Date	String	Variable (to be parsed)	No	
	CustId	String	Variable	No	

- Output Schema:

Same as Input Schema (in-memory DataFrame)

- Processing / Transformation Logic:

1. Use AWS Glue DynamicFrame or Spark DataFrame reader to read CSV files
2. Enable schema inference or apply explicit schema definition
3. Set header option to true
4. Set delimiter to comma
5. Enable multiLine option if records may span multiple lines
6. Load all files matching pattern `s3://adif-sdlc/sdlc\_wizard/orderdata/.csv`
7. Validate that all expected columns (OrderId, ItemName, PricePerUnit, Qty, Date, CustId) are present
8. Store result in DataFrame named df_order_raw

- Validation Rules:

- Verify S3 path exists and is accessible
- Verify at least one CSV file is present in the path

- Verify CSV header contains exactly: OrderId, ItemName, PricePerUnit, Qty, Date, CustId (order may vary)
- Verify at least one data row exists after header
- Error Handling and Logging:
 - Log INFO: "Starting order data ingestion from s3://adif-sdlc/sdlc_wizard/orderdata/"
 - Log INFO: "Number of files found: {count}"
 - Log INFO: "Number of raw records loaded: {count}"
 - If S3 path does not exist: Log ERROR and raise exception "Source path not found: s3://adif-sdlc/sdlc_wizard/orderdata/"
 - If no CSV files found: Log ERROR and raise exception "No CSV files found in source path"
 - If schema mismatch: Log ERROR and raise exception "CSV schema does not match expected columns"
 - If read operation fails: Log ERROR with exception details and halt job
- Runtime Parameters:
 - ``source_order_path`: s3://adif-sdlc/sdlc_wizard/orderdata/`
 - ``csv_delimiter`: `,``
 - ``csv_header`: `true``
 - ``csv_infer_schema`: `true``
- Operational Notes:
 - Use Glue DynamicFrame for better integration with Glue Data Catalog
 - Consider using pushdown predicates if filtering is needed at read time
 - Monitor CloudWatch metrics for read throughput and data volume
 - Date field will require parsing in subsequent transformation steps
-

TR-CLEAN-001: Technical Implementation for Customer Data Cleaning

- Related Functional Requirement ID(s): FR-CLEAN-001
- Objective:

Implement data quality operations to remove null values (both string "Null" and actual NULL) and duplicate records from customer dataset.

- Source Details:
 - Source Type: In-memory Spark DataFrame
 - Source Name: `df_customer_raw` (from TR-INGEST-001)
- Target Details:

- Target Type: In-memory Spark DataFrame
- Target Name: df_customer_clean

- Input Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	CustId	String	Variable	Yes (before cleaning)	
	Name	String	Variable	Yes (before cleaning)	
	EmailId	String	Variable	Yes (before cleaning)	
	Region	String	Variable	Yes (before cleaning)	

- Output Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	CustId	String	Variable	No	
	Name	String	Variable	No	
	EmailId	String	Variable	No	
	Region	String	Variable	No	

- Processing / Transformation Logic:

1. Filter out records where any column contains actual NULL value
2. Filter out records where any column contains string "Null" (case-insensitive match)
3. Apply deduplication by comparing all columns (CustId, Name, EmailId, Region)
4. Retain first occurrence of duplicate records (or use deterministic ordering if available)
5. Store result in DataFrame named df_customer_clean

- Validation Rules:

- After null removal, verify at least one record remains (log warning if zero)
- After deduplication, log count of removed duplicates
- Verify all remaining records have non-null, non-"Null" values in all columns

- Error Handling and Logging:

- Log INFO: "Starting customer data cleaning"
- Log INFO: "Records before null removal: {count}"
- Log INFO: "Records after null removal: {count}"
- Log INFO: "Records removed due to nulls: {count}"
- Log INFO: "Records before deduplication: {count}"
- Log INFO: "Records after deduplication: {count}"
- Log INFO: "Duplicate records removed: {count}"

- If all records removed: Log WARNING "All customer records removed during cleaning; proceeding with empty dataset"
- If cleaning operation fails: Log ERROR with exception details and halt job
- Runtime Parameters:
- ``null_string_values``: ["Null", "null", "NULL"]
- Operational Notes:
- Use Spark SQL functions: ``isNull()``, ``isNotNull()``, ``lower()``, ``isin()``
- Use ``dropDuplicates()`` method for deduplication
- Consider caching DataFrame if used multiple times downstream
- Monitor memory usage for large datasets
-

TR-CLEAN-002: Technical Implementation for Order Data Cleaning

- Related Functional Requirement ID(s): FR-CLEAN-002
- Objective:

Implement data quality operations to remove null values (both string "Null" and actual NULL) and duplicate records from order dataset.

- Source Details:
- Source Type: In-memory Spark DataFrame
- Source Name: df_order_raw (from TR-INGEST-002)
- Target Details:
- Target Type: In-memory Spark DataFrame
- Target Name: df_order_clean
- Input Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	OrderId	String	Variable	Yes (before cleaning)	
	ItemName	String	Variable	Yes (before cleaning)	
	PricePerUnit	Decimal	10,2	Yes (before cleaning)	
	Qty	Integer	-	Yes (before cleaning)	
	Date	String	Variable	Yes (before cleaning)	
	CustId	String	Variable	Yes (before cleaning)	

- Output Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	OrderId	String	Variable	No	
	ItemName	String	Variable	No	
	PricePerUnit	Decimal	10,2	No	
	Qty	Integer	-	No	
	Date	Date	-	No	
	CustId	String	Variable	No	

- Processing / Transformation Logic:

1. Filter out records where any column contains actual NULL value
2. Filter out records where any column contains string "Null" (case-insensitive match)
3. Parse Date column from string to Date type using multiple format attempts:
 - Try YYYY-MM-DD format
 - Try MM/DD/YYYY format
 - Try DD-MM-YYYY format
 - If all fail, treat as null and remove record
4. Cast PricePerUnit to Decimal(10,2) if not already typed
5. Cast Qty to Integer if not already typed
6. Apply deduplication by comparing all columns (OrderId, ItemName, PricePerUnit, Qty, Date, CustId)
7. Retain first occurrence of duplicate records (or use deterministic ordering if available)
8. Store result in DataFrame named df_order_clean

- Validation Rules:

- After null removal, verify at least one record remains (log warning if zero)
- After date parsing, verify all Date values are valid dates
- After type casting, verify PricePerUnit and Qty are numeric
- After deduplication, log count of removed duplicates
- Verify all remaining records have non-null, non-"Null" values in all columns

- Error Handling and Logging:

- Log INFO: "Starting order data cleaning"
- Log INFO: "Records before null removal: {count}"
- Log INFO: "Records after null removal: {count}"
- Log INFO: "Records removed due to nulls: {count}"
- Log INFO: "Records with unparseable dates: {count}"
- Log INFO: "Records before deduplication: {count}"

- Log INFO: "Records after deduplication: {count}"
- Log INFO: "Duplicate records removed: {count}"
- If all records removed: Log WARNING "All order records removed during cleaning; proceeding with empty dataset"
- If cleaning operation fails: Log ERROR with exception details and halt job
- Runtime Parameters:
 - ``null_string_values``: ["Null", "null", "NULL"]
 - ``date_formats``: ["yyyy-MM-dd", "MM/dd/yyyy", "dd-MM-yyyy"]
- Operational Notes:
 - Use Spark SQL functions: ``isNull()``, ``isNotNull()``, ``lower()``, ``isin()``, ``to_date()``, ``coalesce()``
 - Use ``dropDuplicates()`` method for deduplication
 - Consider caching DataFrame if used multiple times downstream
 - Monitor memory usage for large datasets
 - Date parsing may require custom UDF if standard formats fail
-

TR-CATALOG-001: Technical Implementation for Customer Catalog Registration

- Related Functional Requirement ID(s): FR-CATALOG-001
- Objective:

Write cleaned customer data to S3 in Parquet format and register as a table in AWS Glue Data Catalog for query access via Athena.

- Source Details:
 - Source Type: In-memory Spark DataFrame
 - Source Name: df_customer_clean (from TR-CLEAN-001)
- Target Details:
 - Target Type: AWS Glue Data Catalog Table
 - Target Database: gen_ai_poc_databrickscoe
 - Target Table: sdlc_wizard_customer
 - Target S3 Path: s3://adif-sdlc/curated/sdlc_wizard/customer/
 - Target Format: Parquet
 - Write Mode: Overwrite
- Input Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
--	-------------	-----------	------------------------	----------	--

	-----	-----	-----	-----	
	CustId	String	Variable	No	
	Name	String	Variable	No	
	EmailId	String	Variable	No	
	Region	String	Variable	No	

- Output Schema:

Same as Input Schema

- Processing / Transformation Logic:

1. Write df_customer_clean to S3 path s3://adif-sdlc/curated/sdlc_wizard/customer/ in Parquet format
2. Use overwrite mode to replace existing data
3. Enable Parquet compression (Snappy by default)
4. Register or update table `sdlc\_wizard\_customer` in database `gen\_ai\_poc\_databrickscoe`
5. Update catalog metadata with current S3 location and schema
6. Set table properties to indicate Parquet format

- Validation Rules:

- Verify database `gen\_ai\_poc\_databrickscoe` exists before write operation
- Verify S3 write completes successfully
- Verify catalog registration completes successfully
- Verify table is queryable via Athena after registration

- Error Handling and Logging:

- Log INFO: "Writing customer data to s3://adif-sdlc/curated/sdlc_wizard/customer/"
- Log INFO: "Number of records written: {count}"
- Log INFO: "Registering table gen_ai_poc_databrickscoe.sdlc_wizard_customer"
- If database does not exist: Log ERROR and raise exception "Database gen_ai_poc_databrickscoe does not exist"
- If S3 write fails: Log ERROR with exception details and halt job
- If catalog registration fails: Log ERROR with exception details and halt job

- Runtime Parameters:

- `target\_customer\_s3\_path`: s3://adif-sdlc/curated/sdlc_wizard/customer/
- `target\_database`: gen_ai_poc_databrickscoe
- `target\_customer\_table`: sdlc_wizard_customer
- `write\_mode`: overwrite
- `file\_format`: parquet

- ``compression``: snappy
- Operational Notes:
 - Use Glue DynamicFrame ``write_dynamic_frame.from_catalog()`` or Spark ``write.format("parquet").save()``
 - Use Glue ``getSink()`` method for catalog integration
 - Consider partitioning by Region if query patterns support it (not specified in FRD)
 - Monitor S3 storage costs and file sizes
-

TR-CATALOG-002: Technical Implementation for Order Catalog Registration

- Related Functional Requirement ID(s): FR-CATALOG-002
- Objective:

Write cleaned order data to S3 in Parquet format and register as a table in AWS Glue Data Catalog for query access via Athena.

- Source Details:
 - Source Type: In-memory Spark DataFrame
 - Source Name: df_order_clean (from TR-CLEAN-002)
- Target Details:
 - Target Type: AWS Glue Data Catalog Table
 - Target Database: gen_ai_poc_databrickscoe
 - Target Table: sdlc_wizard_order
 - Target S3 Path: s3://adif-sdlc/curated/sdlc_wizard/order/
 - Target Format: Parquet
 - Write Mode: Overwrite
- Input Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	OrderId	String	Variable	No	
	ItemName	String	Variable	No	
	PricePerUnit	Decimal	10,2	No	
	Qty	Integer	-	No	
	Date	Date	-	No	
	CustId	String	Variable	No	

- Output Schema:

Same as Input Schema

- Processing / Transformation Logic:

1. Write df_order_clean to S3 path s3://adif-sdlc/curated/sdlc_wizard/order/ in Parquet format
2. Use overwrite mode to replace existing data
3. Enable Parquet compression (Snappy by default)
4. Register or update table `sdlc\_wizard\_order` in database `gen\_ai\_poc\_databricksco`
5. Update catalog metadata with current S3 location and schema
6. Set table properties to indicate Parquet format

- Validation Rules:

- Verify database `gen\_ai\_poc\_databricksco` exists before write operation
- Verify S3 write completes successfully
- Verify catalog registration completes successfully
- Verify table is queryable via Athena after registration

- Error Handling and Logging:

- Log INFO: "Writing order data to s3://adif-sdlc/curated/sdlc_wizard/order/"
- Log INFO: "Number of records written: {count}"
- Log INFO: "Registering table gen_ai_poc_databricksco.sdlc_wizard_order"
- If database does not exist: Log ERROR and raise exception "Database gen_ai_poc_databricksco does not exist"
- If S3 write fails: Log ERROR with exception details and halt job
- If catalog registration fails: Log ERROR with exception details and halt job

- Runtime Parameters:

- `target\_order\_s3\_path`: s3://adif-sdlc/curated/sdlc_wizard/order/
- `target\_database`: gen_ai_poc_databricksco
- `target\_order\_table`: sdlc_wizard_order
- `write\_mode`: overwrite
- `file\_format`: parquet
- `compression`: snappy

- Operational Notes:

- Use Glue DynamicFrame `write\_dynamic\_frame.from\_catalog()` or Spark `write.format("parquet").save()`
- Use Glue `getSink()` method for catalog integration
- Consider partitioning by Date if query patterns support it (not specified in FRD)

- Monitor S3 storage costs and file sizes

-

TR-SCD2-001: Technical Implementation for SCD Type 2 Order Summary

- Related Functional Requirement ID(s): FR-SCD2-001

- Objective:

Implement Slowly Changing Dimension Type 2 logic using Apache Hudi to maintain historical order summary records with change tracking via IsActive, StartDate, EndDate, and OpTs fields.

- Source Details:
 - Source Type: In-memory Spark DataFrames
 - Source Name 1: df_customer_clean (from TR-CLEAN-001)
 - Source Name 2: df_order_clean (from TR-CLEAN-002)
 - Existing Hudi Table: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/ (if exists)
- Target Details:
 - Target Type: Apache Hudi Table (Copy-on-Write)
 - Target Database: gen_ai_poc_databricksco
 - Target Table: ordersummary
 - Target S3 Path: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
 - Target Format: Hudi
 - Write Mode: Upsert
- Input Schema (Customer):

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	CustId	String	Variable	No	
	Name	String	Variable	No	
	EmailId	String	Variable	No	
	Region	String	Variable	No	

- Input Schema (Order):

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	OrderId	String	Variable	No	
	ItemName	String	Variable	No	
	PricePerUnit	Decimal	10,2	No	
	Qty	Integer	-	No	

	Date	Date	-	No	
	CustId	String	Variable	No	

• Output Schema (Order Summary):

Column Name	Data Type	Length/Precision/Scale	Nullable	Source Mapping	
-----	-----	-----	-----	-----	
CustId	String	Variable	No	Direct map from Customer.CustId	
Name	String	Variable	No	Direct map from Customer.Name	
EmailId	String	Variable	No	Direct map from Customer.EmailId	
Region	String	Variable	No	Direct map from Customer.Region	
OrderId	String	Variable	No	Direct map from Order.OrderId	
ItemName	String	Variable	No	Direct map from Order.ItemName	
PricePerUnit	Decimal	10,2	No	Direct map from Order.PricePerUnit	
Qty	Integer	-	No	Direct map from Order.Qty	
Date	Date	-	No	Direct map from Order.Date	
IsActive	Boolean	-	No	Derived: true for new/current records, false for superseded	
StartDate	Timestamp	-	No	Derived: current processing timestamp for new records	
EndDate	Timestamp	-	Yes	Derived: null for active records, processing timestamp for superseded	
OpTs	Timestamp	-	No	Derived: current processing timestamp	

• Processing / Transformation Logic:

1. Perform inner join between df_customer_clean and df_order_clean on CustId

2. Add SCD2 metadata columns to joined dataset:

• IsActive = true

• StartDate = current_timestamp()

• EndDate = null

• OpTs = current_timestamp()

3. Define Hudi configuration:

• Table type: COPY_ON_WRITE

• Record key: Composite of CustId + OrderId (concatenated with delimiter)

• Precombine field: OpTs

• Partition path: None (or Date-based if specified)

• Table name: ordersummary

4. Read existing Hudi table from s3://adif-sdlc/curated/sdlc_wizard/ordersummary/ if exists

5. For each record in new dataset:

• Check if matching record exists in Hudi table (match on CustId + OrderId)

• If match found and any business attribute differs (Name, EmailId, Region, ItemName, PricePerUnit, Qty, Date):

- Update existing record: Set IsActive = false, EndDate = current_timestamp()
 - Insert new record with IsActive = true, StartDate = current_timestamp(), EndDate = null
 - If no match found:
 - Insert new record with IsActive = true, StartDate = current_timestamp(), EndDate = null
6. Write updated dataset to Hudi table using upsert operation
7. Register or update table `ordersummary` in database `gen\_ai\_poc\_databrickscoe`
- Validation Rules:
 - Verify join produces at least one record (log warning if zero)
 - Verify Hudi record key is unique within new dataset
 - Verify OpTs is populated for all records
 - Verify IsActive is boolean type
 - After write, verify Hudi table is readable and queryable
 - Error Handling and Logging:
 - Log INFO: "Starting SCD2 processing for order summary"
 - Log INFO: "Joined customer-order records: {count}"
 - Log INFO: "Reading existing Hudi table from s3://adif-sdlc/curated/sdlc_wizard/ordersummary/"
 - Log INFO: "Existing Hudi records: {count}" (or "No existing table found" if first run)
 - Log INFO: "New records to insert: {count}"
 - Log INFO: "Records to update (close): {count}"
 - Log INFO: "Writing to Hudi table"
 - Log INFO: "Hudi write completed successfully"
 - Log INFO: "Registering table gen_ai_poc_databrickscoe.ordersummary"
 - If join produces no records: Log WARNING "No records produced from customer-order join; skipping SCD2 write"
 - If Hudi write fails: Log ERROR with exception details and halt job
 - If catalog registration fails: Log ERROR with exception details and halt job
 - Runtime Parameters:
 - `target\_ordersummary\_s3\_path`: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
 - `target\_database`: gen_ai_poc_databrickscoe
 - `target\_ordersummary\_table`: ordersummary
 - `hudi\_table\_type`: COPY_ON_WRITE
 - `hudi\_record\_key`: CustId,OrderId
 - `hudi\_precombine\_field`: OpTs
 - `hudi\_table\_name`: ordersummary

- `hoodie\_operation`: upsert
- Operational Notes:
- Use Hudi Spark DataSource API for write operations
- Hudi configuration options:

```

```
hoodie.table.name = ordersummary
hoodie.datasource.write.recordkey.field = CustId,OrderId
hoodie.datasource.write.precombine.field = OpTs
hoodie.datasource.write.table.type = COPY_ON_WRITE
hoodie.datasource.write.operation = upsert
hoodie.datasource.hive_sync.enable = true
hoodie.datasource.hive_sync.database = gen_ai_poc_databrickscoe
hoodie.datasource.hive_sync.table = ordersummary
hoodie.datasource.hive_sync.partition_extractor_class = org.apache.hudi.hive.NonPartitionedExtractor
```

```

- Consider enabling Hudi metadata table for query performance
- Monitor Hudi commit timeline and compaction operations
- SCD2 logic requires custom implementation to handle IsActive flag updates (Hudi does not natively support SCD2)
- Alternative approach: Use Hudi's built-in upsert and implement SCD2 logic via merge-on-read with custom logic
- Note: Full SCD2 implementation with IsActive flag management requires additional logic beyond standard Hudi upsert. This may involve:
 1. Reading existing active records
 2. Comparing with new records to detect changes
 3. Marking changed records as inactive (IsActive=false, EndDate=now)
 4. Inserting new versions as active (IsActive=true, StartDate=now)
 5. Writing both updates and inserts to Hudi table
-

TR-INCR-001: Technical Implementation for Incremental Customer Processing

- Related Functional Requirement ID(s): FR-INCR-001
- Objective:

Implement incremental processing logic to identify changed customer records and trigger SCD2 updates only for affected order summaries, optimizing performance by avoiding full reprocessing.

- Source Details:
- Source Type: In-memory Spark DataFrames
- Source Name 1: df_customer_clean (current run, from TR-CLEAN-001)
- Source Name 2: df_customer_previous (previous run baseline)
- Source Name 3: df_order_clean (from TR-CLEAN-002)
- Existing Hudi Table: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/

- Target Details:
- Target Type: Apache Hudi Table (Copy-on-Write)
- Target Database: gen_ai_poc_databrickscoee
- Target Table: ordersummary
- Target S3 Path: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
- Target Format: Hudi
- Write Mode: Upsert (incremental)

- Input Schema (Current Customer):

Same as TR-CLEAN-001 output

- Input Schema (Previous Customer):

Same as TR-CLEAN-001 output

- Input Schema (Order):

Same as TR-CLEAN-002 output

- Output Schema:

Same as TR-SCD2-001 output

- Processing / Transformation Logic:

1. Load previous customer baseline from checkpoint location (e.g., s3://adif-sdlc/checkpoints/customer_baseline/)
2. If no previous baseline exists, treat all current records as new and proceed with full SCD2 processing (TR-SCD2-001)
3. If previous baseline exists:
 - Perform full outer join between df_customer_clean (current) and df_customer_previous on CustId
 - Identify changed records where any attribute differs (Name, EmailId, Region)
 - Identify new records (present in current, absent in previous)
 - Identify deleted records (present in previous, absent in current) - log for informational purposes
4. Extract list of changed/new CustId values
5. Filter df_order_clean to include only orders where CustId is in the changed/new list

6. Execute SCD2 logic (TR-SCD2-001) using filtered customer and order datasets
7. Write current customer dataset as new baseline to checkpoint location
8. Merge incremental changes into existing Hudi ordersummary table

- Validation Rules:

- Verify checkpoint location is accessible
- Verify changed customer count is logged
- If no changes detected, skip SCD2 processing and log informational message
- Verify baseline write completes successfully for next run

- Error Handling and Logging:

- Log INFO: "Starting incremental customer processing"
- Log INFO: "Loading previous customer baseline from checkpoint"
- If no baseline found: Log INFO "No previous baseline found; treating all records as new"
- If baseline found: Log INFO "Previous baseline loaded: {count} records"
- Log INFO: "Current customer records: {count}"
- Log INFO: "Changed customer records: {count}"
- Log INFO: "New customer records: {count}"
- Log INFO: "Deleted customer records: {count}"
- If no changes: Log INFO "No customer changes detected; skipping SCD2 processing"
- Log INFO: "Filtered orders for changed customers: {count}"
- Log INFO: "Executing SCD2 logic for incremental changes"
- Log INFO: "Writing current customer baseline to checkpoint"
- If checkpoint write fails: Log ERROR with exception details and halt job
- If incremental SCD2 processing fails: Log ERROR with exception details and halt job

- Runtime Parameters:

- ``checkpoint_customer_baseline_path``: s3://adif-sdlc/checkpoints/customer_baseline/
- ``incremental_mode``: true

- Operational Notes:

- Checkpoint baseline should be stored in Parquet format for efficient read/write
- Use Spark DataFrame ``except()`` or ``subtract()`` methods to identify changes
- Consider using hash-based change detection for performance (hash all columns, compare hashes)
- Incremental processing assumes order data is cumulative (all orders are present in each run)
- If order data is also incremental, additional logic is required to filter orders by date range
- Monitor checkpoint storage size and implement retention policy if needed

-

TR-AGG-001: Technical Implementation for Customer Aggregate Spend

- Related Functional Requirement ID(s): FR-AGG-001

- Objective:

Calculate total spending per customer by aggregating order amounts ($\text{PricePerUnit} \times \text{Qty}$) grouped by customer Name and order Date, and store results in an analytics table for BI consumption.

- Source Details:

- Source Type: In-memory Spark DataFrames
- Source Name 1: df_customer_clean (from TR-CLEAN-001)
- Source Name 2: df_order_clean (from TR-CLEAN-002)

- Target Details:

- Target Type: AWS Glue Data Catalog Table
- Target Database: gen_ai_poc_databrickscoe
- Target Table: customeragggregatespend
- Target S3 Path: s3://adif-sdlc/analytics/customeragggregatespend/
- Target Format: Parquet
- Write Mode: Overwrite

- Input Schema (Customer):

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	CustId	String	Variable	No	
	Name	String	Variable	No	
	EmailId	String	Variable	No	
	Region	String	Variable	No	

- Input Schema (Order):

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	OrderId	String	Variable	No	
	ItemName	String	Variable	No	
	PricePerUnit	Decimal	10,2	No	
	Qty	Integer	-	No	
	Date	Date	-	No	
	CustId	String	Variable	No	

• Output Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	Source Mapping	
	-----	-----	-----	-----	-----	
	Name	String	Variable	No	Direct map from Customer.Name	
	TotalAmount	Decimal	12,2	No	Derived: SUM(Order.PricePerUnit × Order.Qty)	
	Date	Date	-	No	Direct map from Order.Date	

• Processing / Transformation Logic:

1. Perform inner join between df_customer_clean and df_order_clean on CustId
2. Calculate line-level amount: Create new column `LineAmount = PricePerUnit Qty`
3. Group by Name and Date
4. Aggregate: `TotalAmount = SUM(LineAmount)`
5. Select final columns: Name, TotalAmount, Date
6. Write aggregated dataset to S3 path s3://adif-sdlc/analytics/customeraggregatespend/ in Parquet format
7. Use overwrite mode to replace existing data
8. Register or update table `customeraggregatespend` in database `gen\_ai\_poc\_databrickscoe`

• Validation Rules:

- Verify join produces at least one record (log warning if zero)
- Verify TotalAmount is non-negative (log warning if negative values found)
- Verify TotalAmount precision is Decimal(12,2)
- Verify aggregation produces at least one group (log warning if zero)
- Verify S3 write completes successfully
- Verify catalog registration completes successfully

• Error Handling and Logging:

- Log INFO: "Starting customer aggregate spend calculation"
- Log INFO: "Joined customer-order records: {count}"
- Log INFO: "Calculating line-level amounts"
- Log INFO: "Grouping by Name and Date"
- Log INFO: "Aggregated groups: {count}"
- Log INFO: "Writing aggregated data to s3://adif-sdlc/analytics/customeraggregatespend/"
- Log INFO: "Number of aggregated records written: {count}"
- Log INFO: "Registering table gen_ai_poc_databrickscoe.customeraggregatespend"
- If join produces no records: Log WARNING "No records produced from customer-order join; creating empty table"

- If negative TotalAmount found: Log WARNING "Negative TotalAmount values detected: {count} records"
- If aggregation fails: Log ERROR with exception details and halt job
- If S3 write fails: Log ERROR with exception details and halt job
- If catalog registration fails: Log ERROR with exception details and halt job
- Runtime Parameters:
 - `target\_aggregate\_s3\_path`: s3://adif-sdlc/analytics/customeraggregatespend/
 - `target\_database`: gen_ai_poc_databricksco
 - `target\_aggregate\_table`: customeraggregatespend
 - `write\_mode`: overwrite
 - `file\_format`: parquet
 - `compression`: snappy
- Operational Notes:
 - Use Spark SQL `groupBy()` and `agg()` functions
 - Use `sum()` aggregation function for TotalAmount
 - Consider partitioning by Date for query performance (not specified in FRD)
 - Monitor aggregation performance for large datasets
 - Consider using broadcast join if customer dataset is small
 - Validate that PricePerUnit and Qty are numeric before multiplication
-

3. Runtime Strategy Notes

Authoring Language

- Primary Language: PySpark (Python with Spark API)
- Alternative: Scala with Spark API (if team preference or performance requirements dictate)
- Glue Version: AWS Glue 4.0
- Spark Version: 3.3.0 (bundled with Glue 4.0)
- Python Version: 3.10 (bundled with Glue 4.0)

Execution Strategy

- Job Type: AWS Glue ETL Job (Spark-based)
- Worker Type: G.1X or G.2X (standard workers)
- Number of Workers: Minimum 2, scale based on data volume
- Max Capacity: Not applicable (using worker type configuration)
- Timeout: 2880 minutes (48 hours) - adjust based on expected runtime

- Max Retries: 0 (handle retries at orchestration layer)
- Job Bookmark: Disabled (incremental logic handled via checkpoint mechanism)
- Continuous Logging: Enabled (CloudWatch Logs)
- Spark UI: Enabled for debugging

Glue Job Type Expectations

- ETL Job: Standard Glue Spark job for data processing
- Script Location: s3:///scripts/customer_order_pipeline.py
- Temporary Directory: s3:///temp/
- Job Parameters:
 - `--source_customer_path` : s3://adif-sdlc/sdlc_wizard/customerdata/`
 - `--source_order_path` : s3://adif-sdlc/sdlc_wizard/orderdata/`
 - `--target_database` : gen_ai_poc_databrickscoe`
 - `--target_customer_table` : sdlc_wizard_customer`
 - `--target_order_table` : sdlc_wizard_order`
 - `--target_ordersummary_table` : ordersummary`
 - `--target_aggregate_table` : customeraggregatespend`
 - `--checkpoint_path` : s3://adif-sdlc/checkpoints/customer_baseline/`
 - `--incremental_mode` : true/false`
 - `--enable_spark_ui` : true`
 - `--enable_continuous_cloudwatch_log` : true`
 - `--enable_metrics` : true`
 -

4. Data Design Details

Source Paths / Source Systems

Source ID	Source Type	Source Path	Format	Schema
-----	-----	-----	-----	-----
SRC-CUST-001	AWS S3 CSV	s3://adif-sdlc/sdlc_wizard/customerdata/	CSV	CustId, Name, EmailId, Region
SRC-ORD-001	AWS S3 CSV	s3://adif-sdlc/sdlc_wizard/orderdata/	CSV	OrderId, ItemName, PricePerUnit, Qty, Date, CustId

- Note: These are the ORIGINAL sources from the FRD. No intermediate staging tables or alternative sources are introduced.

Target Paths / Target Tables

Target Type	Target Path	Format	Table Name	Database
-----	-----	-----	-----	-----

	Glue Catalog Table	s3://adif-sdlc/curated/sdlc_wizard/customer/	Parquet	sdlc_wizard_customer	gen_ai_poc_databricksco
	Glue Catalog Table	s3://adif-sdlc/curated/sdlc_wizard/order/	Parquet	sdlc_wizard_order	gen_ai_poc_databricksco
001	Hudi Table	s3://adif-sdlc/curated/sdlc_wizard/ordersummary/	Hudi (CoW)	ordersummary	gen_ai_poc_databricksco
	Glue Catalog Table	s3://adif-sdlc/analytics/customeraggregatespend/	Parquet	customeraggregatespend	gen_ai_poc_databricksco

Catalog Registration Expectations

- All target tables must be registered in AWS Glue Data Catalog database `gen\_ai\_poc\_databricksco`
- Catalog registration should occur immediately after S3 write operation
- Use Glue `getSink()` method or Hudi Hive Sync for automatic catalog updates
- Verify table metadata (schema, location, format) is accurate after registration
- Enable Athena query access for all registered tables

Partitioning Expectations

- Customer Table: No partitioning specified in FRD (implement if query patterns support Region-based partitioning)
- Order Table: No partitioning specified in FRD (consider Date-based partitioning for query performance)
- Order Summary Hudi Table: No partitioning specified in FRD (Hudi supports partitioning via partition path configuration)
- Customer Aggregate Spend Table: No partitioning specified in FRD (consider Date-based partitioning for query performance)
- Recommendation: Implement Date-based partitioning for Order and Aggregate tables if query patterns frequently filter by date range.

File Formats / Table Formats

- Customer Table: Parquet with Snappy compression
- Order Table: Parquet with Snappy compression
- Order Summary Table: Apache Hudi Copy-on-Write (CoW) format
- Customer Aggregate Spend Table: Parquet with Snappy compression
- Checkpoint Baseline: Parquet with Snappy compression

Update / Overwrite / Append Behavior

	Target Table	Write Mode	Behavior	
	-----	-----	-----	
	sdlc_wizard_customer	Overwrite	Replace all existing data with current cleaned dataset	
	sdlc_wizard_order	Overwrite	Replace all existing data with current cleaned dataset	
	ordersummary	Upsert	Insert new records and update existing records based on SCD2 logic	
	customeraggregatespend	Overwrite	Replace all existing data with current aggregated dataset	

	customer_baseline (checkpoint)	Overwrite	Replace previous baseline with current customer dataset	
--	--------------------------------	-----------	---	--

- Note: Hudi upsert mode enables efficient incremental updates without full table rewrites.

-

5. Processing Details

Data Quality Rules

1. Null Value Removal:

- Remove records where any column contains actual NULL value
- Remove records where any column contains string "Null" (case-insensitive: "Null", "null", " NULL")
- Apply to both customer and order datasets

2. Duplicate Removal:

- Identify duplicates by comparing all columns in each dataset
- Retain first occurrence of duplicate records
- Apply to both customer and order datasets

3. Data Type Validation:

- Validate PricePerUnit is numeric (Decimal)
- Validate Qty is numeric (Integer)
- Validate Date is parseable to Date type
- Reject records that fail type validation

4. Business Rule Validation:

- Validate PricePerUnit ≥ 0 (log warning if negative)
- Validate Qty > 0 (log warning if zero or negative)
- Validate Date is not in future (log warning if future date)

Deduplication Rules

- Method: Spark `dropDuplicates()` function
- Comparison: All columns in dataset
- Retention: First occurrence (based on DataFrame row order)
- Logging: Log count of duplicates removed
- Performance: Consider using hash-based deduplication for large datasets

Join Rules

1. Customer-Order Join (for SCD2):

- Join Type: Inner join
- Join Key: CustId
- Result: All orders with matching customer records
- Handling: Log warning if join produces zero records

2. Customer-Order Join (for Aggregation):

- Join Type: Inner join
- Join Key: CustId
- Result: All orders with matching customer records
- Handling: Log warning if join produces zero records

3. Current-Previous Customer Join (for Incremental):

- Join Type: Full outer join
- Join Key: CustId
- Result: All customers from both datasets with change indicators
- Handling: Identify new, changed, and deleted records

Aggregation Rules

1. Customer Aggregate Spend:

- Grouping Columns: Name, Date
- Aggregation Function: $\text{SUM}(\text{PricePerUnit} \times \text{Qty})$
- Output Column: TotalAmount (Decimal 12,2)
- Null Handling: Exclude records with null PricePerUnit or Qty before aggregation
- Validation: Verify TotalAmount ≥ 0 (log warning if negative)

SCD / History Tracking Rules

1. SCD Type 2 Implementation:

- Record Key: Composite of CustId + OrderId
- Change Detection: Compare all business attributes (Name, EmailId, Region, ItemName, PricePerUnit, Qty, Date)
- Active Record Indicator: IsActive (Boolean)
- Effective Date Tracking: StartDate (Timestamp), EndDate (Timestamp)
- Operation Timestamp: OpTs (Timestamp)

2. Change Handling:

- New Record: Insert with IsActive=true, StartDate=now, EndDate=null, OpTs=now
- Changed Record:
- Update existing: Set IsActive=false, EndDate=now

- Insert new: Set IsActive=true, StartDate=now, EndDate=null, OpTs=now
- Unchanged Record: No action

3. Hudi Configuration:

- Table Type: COPY_ON_WRITE
- Record Key Field: CustId,OrderId (composite)
- Precombine Field: OpTs
- Operation: Upsert
- Hive Sync: Enabled for catalog integration

Null Handling

1. String "Null" Handling:

- Treat string "Null" (case-insensitive) as missing data
- Remove records containing "Null" string in any column
- Apply during data cleaning phase

2. Actual NULL Handling:

- Remove records containing actual NULL values in any column
- Apply during data cleaning phase

3. Post-Cleaning NULL Handling:

- After cleaning, all columns should be non-nullable
- If NULL values appear in downstream processing, log error and investigate

Type Casting Expectations

1. Customer Data:

- CustId: String (no casting required)
- Name: String (no casting required)
- EmailId: String (no casting required)
- Region: String (no casting required)

2. Order Data:

- OrderId: String (no casting required)
- ItemName: String (no casting required)
- PricePerUnit: Cast to Decimal(10,2) if read as String
- Qty: Cast to Integer if read as String
- Date: Parse from String to Date using multiple format attempts
- CustId: String (no casting required)

3. Derived Columns:

- LineAmount: Decimal(12,2) (calculated as PricePerUnit × Qty)
- TotalAmount: Decimal(12,2) (aggregated sum of LineAmount)
- IsActive: Boolean
- StartDate: Timestamp (current_timestamp())
- EndDate: Timestamp (nullable)
- OpTs: Timestamp (current_timestamp())
-

6. Traceability Matrix

Requirement ID(s)	Original Source	Technical Source	Description
-----	-----	-----	-----
EST-001	s3://adif-sdlc/sdlc_wizard/customerdata/	s3://adif-sdlc/sdlc_wizard/customerdata/	Customer
EST-002	s3://adif-sdlc/sdlc_wizard/orderdata/	s3://adif-sdlc/sdlc_wizard/orderdata/	Order
AN-001	df_customer_raw	df_customer_raw	Customer
AN-002	df_order_raw	df_order_raw	Order
ATALOG-001	df_customer_clean	df_customer_clean	Customer
ATALOG-002	df_order_clean	df_order_clean	Order
D2-001	df_customer_clean, df_order_clean	df_customer_clean, df_order_clean	SCD2
R-001	df_customer_clean, df_customer_previous, df_order_clean	df_customer_clean, df_customer_previous, df_order_clean	Incremental
G-001	df_customer_clean, df_order_clean	df_customer_clean, df_order_clean	Customer

- Note: All technical sources match the original sources from the FRD exactly. No intermediate staging tables or alternative sources have been introduced.

•

7. ETL Mapping Requirements

Source Entity: Customer CSV (s3://adif-sdlc/sdlc_wizard/customerdata/)

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	CustId	String	Variable	Yes (before cleaning)	
	Name	String	Variable	Yes (before cleaning)	
	EmailId	String	Variable	Yes (before cleaning)	
	Region	String	Variable	Yes (before cleaning)	

Source Entity: Order CSV (s3://adif-sdlc/sdlc_wizard/orderdata/)

	Column Name	Data Type	Length/Precision/Scale	Nullable	
--	-------------	-----------	------------------------	----------	--

	-----	-----	-----	-----	
	OrderId	String	Variable	Yes (before cleaning)	
	ItemName	String	Variable	Yes (before cleaning)	
	PricePerUnit	Decimal	10,2	Yes (before cleaning)	
	Qty	Integer	-	Yes (before cleaning)	
	Date	String/Date	Variable	Yes (before cleaning)	
	CustId	String	Variable	Yes (before cleaning)	

Target Entity: sdlc_wizard_customer (gen_ai_poc_databrickscoe.sdlc_wizard_customer)

	Column Name	Data Type	Length/Precision/Scale	Nullable	Source Mapping	
	-----	-----	-----	-----	-----	
	CustId	String	Variable	No	Direct map from Customer CSV.CustId	
	Name	String	Variable	No	Direct map from Customer CSV.Name	
	EmailId	String	Variable	No	Direct map from Customer CSV.EmailId	
	Region	String	Variable	No	Direct map from Customer CSV.Region	

Target Entity: sdlc_wizard_order (gen_ai_poc_databrickscoe.sdlc_wizard_order)

	Column Name	Data Type	Length/Precision/Scale	Nullable	Source Mapping	
	-----	-----	-----	-----	-----	
	OrderId	String	Variable	No	Direct map from Order CSV.OrderId	
	ItemName	String	Variable	No	Direct map from Order CSV.ItemName	
	PricePerUnit	Decimal	10,2	No	Direct map from Order CSV.PricePerUnit (cast to Decimal)	
	Qty	Integer	-	No	Direct map from Order CSV.Qty (cast to Integer)	
	Date	Date	-	No	Derived from Order CSV.Date (parsed from String to Date)	
	CustId	String	Variable	No	Direct map from Order CSV.CustId	

Target Entity: ordersummary (gen_ai_poc_databrickscoe.ordersummary)

	Column Name	Data Type	Length/Precision/Scale	Nullable	Source Mapping	
	-----	-----	-----	-----	-----	
	CustId	String	Variable	No	Direct map from sdlc_wizard_customer.CustId	
	Name	String	Variable	No	Direct map from sdlc_wizard_customer.Name	
	EmailId	String	Variable	No	Direct map from sdlc_wizard_customer.EmailId	
	Region	String	Variable	No	Direct map from sdlc_wizard_customer.Region	
	OrderId	String	Variable	No	Direct map from sdlc_wizard_order.OrderId	
	ItemName	String	Variable	No	Direct map from sdlc_wizard_order.ItemName	
	PricePerUnit	Decimal	10,2	No	Direct map from sdlc_wizard_order.PricePerUnit	
	Qty	Integer	-	No	Direct map from sdlc_wizard_order.Qty	
	Date	Date	-	No	Direct map from sdlc_wizard_order.Date	
	IsActive	Boolean	-	No	Derived: true for current records, false for superseded	

	StartDate	Timestamp	-	No	Derived: current_timestamp() for new/updated records	
	EndDate	Timestamp	-	Yes	Derived: null for active, current_timestamp() for superseded	
	OpTs	Timestamp	-	No	Derived: current_timestamp() for all operations	

Target Entity: customeraggregatespend (gen_ai_poc_databrickscoe.customeraggregatespend)

Name	Data Type	Length/Precision/Scale	Nullable	Source Mapping
	-----	-----	-----	-----
	String	Variable	No	Direct map from sdlc_wizard_customer.Name
ount	Decimal	12,2	No	Derived: SUM(sdlc_wizard_order.PricePerUnit × sdlc_wizard_order.Qty) grouped by Na
	Date	-	No	Direct map from sdlc_wizard_order.Date

•

Assumptions

1. Data Type Precision: Exact precision and scale for Decimal types are assumed based on typical business requirements (PricePerUnit: 10,2; TotalAmount: 12,2). If different precision is required, this should be specified.
2. Date Format: The FRD does not specify the exact date format in the Order CSV. The TRD assumes multiple common formats will be attempted during parsing (YYYY-MM-DD, MM/DD/YYYY, DD-MM-YYYY).
3. Hudi Record Key: The FRD does not specify the exact record key for Hudi SCD2 implementation. The TRD assumes a composite key of CustId + OrderId is appropriate for uniquely identifying order summary records.
4. SCD2 Change Detection: The FRD states "all customer and order attributes" should be compared for change detection. The TRD assumes this means all business attributes excluding system-generated fields (IsActive, StartDate, EndDate, OpTs).
5. Incremental Baseline Storage: The FRD does not specify where the previous customer baseline should be stored for incremental processing. The TRD assumes a checkpoint location in S3 (s3://adif-sdlc/checkpoints/customer_baseline/).
6. Catalog Table Storage Location: The FRD does not specify the exact S3 paths for catalog-registered customer and order tables. The TRD assumes paths under s3://adif-sdlc/curated/sdlc_wizard/ for consistency.
7. Partitioning Strategy: The FRD does not specify partitioning requirements. The TRD notes this as an assumption and recommends Date-based partitioning for query performance where applicable.
8. Glue Job Configuration: The FRD does not specify Glue job worker type, DPU allocation, or timeout settings. The TRD assumes standard configurations (G.1X workers, minimum 2 workers, 48-hour timeout).

9. Error Handling Strategy: The FRD does not specify detailed error handling requirements. The TRD assumes standard logging to CloudWatch with job failure on critical errors.

10. Hudi Table Type: The FRD specifies Hudi format but not the table type. The TRD assumes Copy-on-Write (CoW) for simplicity and consistency with typical SCD2 use cases.

11. Aggregation Grouping: The FRD specifies grouping by "Name and Date" for aggregation. The TRD assumes this refers to customer Name and order Date, not a processing date.

12. Full SCD2 Implementation: The FRD requires SCD2 with IsActive flag management. Standard Hudi upsert does not natively support this pattern. The TRD notes that custom logic is required to implement full SCD2 behavior (reading existing records, comparing, marking inactive, inserting new versions).

-

- End of Technical Requirements Document